

Nome: André Machado Silva

Repositório no GitHub: <https://github.com/andrems1/onboarding-LIPAI>

Preparação dos Dados

Semana 7 / Atividade 2

Exercícios — Roteiro Lab2 (Questões 1 a 6)

Questão 1 — Carregar e inspecionar o dataset

Nesta etapa inicial, o dataset é carregado com a biblioteca Pandas e inspecionado com funções como *head()*, *info()* e *describe()* para entender sua estrutura, tipos de dados, quantidade de registros e presença de valores ausentes.

```
import pandas as pd
import numpy as np

# Carregar o dataset
url = 'https://raw.githubusercontent.com/marcelozanchetta/EDS104/main/dados.csv'
df = pd.read_csv(url)

# Inspeccionar
print(df.head())
print(df.info())
print(df.describe())
print("Valores ausentes por coluna:")
print(df.isnull().sum())
```

Questão 2 — Identificar tipos de dados (discretos, contínuos e categóricos)

A classificação correta dos tipos de dados guia todas as etapas de pré-processamento. Dados numéricos contínuos (como altura e temperatura) exigem normalização; dados discretos (como contagens) podem ser usados diretamente; dados categóricos precisam de codificação (Label Encoding ou One-Hot Encoding).

```
# Separar colunas por tipo
colunas_numericas = df.select_dtypes(include=['float64', 'int64']).columns
colunas_categoricas = df.select_dtypes(include=['object', 'category']).columns

print("Colunas numéricas:", list(colunas_numericas))
print("Colunas categóricas:", list(colunas_categoricas))

# Exemplo de identificação manual
# Contínuas: salário, idade (quando tratado como float), temperatura
# Discretas: número de filhos, quantidade de itens
# Categóricas: sexo, estado civil, tipo sanguíneo
```

Questão 3 — Tratamento de valores ausentes

Valores ausentes (NaN) devem ser tratados antes do treinamento do modelo. As estratégias mais comuns são imputação pela média ou mediana (para dados numéricos), pela moda (para categóricos) ou remoção das linhas/colunas com muitos dados faltantes. A escolha depende do percentual de dados ausentes e do impacto no modelo.

```
from sklearn.impute import SimpleImputer

# Imputação pela mediana para colunas numéricas
imputer_num = SimpleImputer(strategy='median')
df[colunas_numericas] = imputer_num.fit_transform(df[colunas_numericas])

# Imputação pela moda para colunas categóricas
imputer_cat = SimpleImputer(strategy='most_frequent')
df[colunas_categoricas] = imputer_cat.fit_transform(df[colunas_categoricas])

print("Valores ausentes após tratamento:")
print(df.isnull().sum())
```

Questão 4 — Normalização dos dados numéricos

A normalização coloca todas as variáveis numéricas em escalas comparáveis, evitando que atributos com valores muito altos (como renda em R\$) dominem o aprendizado em detrimento de atributos com valores pequenos (como idade em anos). Foram aplicados dois métodos: Min-Max Scaling (escala [0,1]) e Z-Score Standardization.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Min-Max Scaling (valores entre 0 e 1)
scaler_minmax = MinMaxScaler()
df_minmax = pd.DataFrame(
    scaler_minmax.fit_transform(df[colunas_numericas]),
    columns=colunas_numericas
)
print("Após Min-Max Scaling:")
print(df_minmax.describe())

# Z-Score Standardization (média=0, desvio=1)
scaler_std = StandardScaler()
df_std = pd.DataFrame(
    scaler_std.fit_transform(df[colunas_numericas]),
    columns=colunas_numericas
)
print("Após Z-Score:")
print(df_std.describe())
```

Questão 5 — Codificação de variáveis categóricas

Algoritmos de ML geralmente requerem entradas numéricas. Variáveis categóricas nominais (sem ordem) são transformadas com One-Hot Encoding, que cria uma coluna binária para cada categoria. Variáveis ordinais (com ordem natural, como 'pequeno', 'médio', 'grande') são tratadas com Label Encoding.

```
from sklearn.preprocessing import LabelEncoder

# One-Hot Encoding para variáveis nominais
df_encoded = pd.get_dummies(df, columns=['sexo', 'estado_civil'], drop_first=True)
```

```
# Label Encoding para variáveis ordinais
le = LabelEncoder()
# Exemplo: coluna 'tamanho' com valores 'pequeno', 'médio', 'grande'
# df['tamanho_cod'] = le.fit_transform(df['tamanho'])

print("Colunas após encoding:")
print(df_encoded.columns.tolist())
print(df_encoded.head())
```

Questão 6 — Divisão em conjuntos de treino e teste

A divisão do dataset em conjuntos de treino e teste é fundamental para avaliar a capacidade de generalização do modelo. O conjunto de treino é usado para ajustar os parâmetros do modelo; o de teste simula dados nunca vistos, permitindo uma avaliação imparcial. A proporção clássica é 80% para treino e 20% para teste. Para datasets pequenos, a validação cruzada (k-fold) é mais indicada.

```
from sklearn.model_selection import train_test_split

# Definir features (X) e alvo (y) – ajustar conforme o dataset
X = df_encoded.drop(columns=['target'])
y = df_encoded['target']

# Divisão 80/20 com aleatoriedade controlada
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Tamanho do treino: {X_train.shape[0]} registros ({X_train.shape[0]/len(X)*100:.1f}%)")
print(f"Tamanho do teste: {X_test.shape[0]} registros ({X_test.shape[0]/len(X)*100:.1f}%)")

# Exemplo de treino com um modelo simples
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"Acurácia no conjunto de teste: {accuracy_score(y_test, y_pred):.4f}")
```

Reflexões sobre o Pré-processamento de Dados

A preparação dos dados é, frequentemente, a etapa mais trabalhosa e crítica em qualquer projeto de Machine Learning. Estima-se que profissionais da área dedicam entre 60% e 80% do tempo total de um projeto a tarefas de coleta, limpeza e transformação de dados.

A distinção entre dados discretos, contínuos e categóricos não é apenas acadêmica — ela determina diretamente quais transformações são aplicáveis e quais algoritmos podem ser utilizados. Aplicar normalização a uma variável categórica ordinal sem o encoding correto, por exemplo, pode introduzir relações numéricas falsas no modelo.

O conceito de overfitting reforça a importância da separação treino/teste: um modelo que 'decora' os dados de treinamento apresenta excelente desempenho interno, mas falha ao generalizar para novos dados. A validação cruzada é uma solução elegante para datasets pequenos, onde sacrificar

20% dos dados para teste pode ser custoso para a qualidade do treinamento.

Por fim, o balanceamento de classes é um aspecto frequentemente subestimado: datasets desbalanceados (onde uma classe tem muito mais exemplos que outra) podem levar modelos a simplesmente ignorar a classe minoritária, que muitas vezes é exatamente o fenômeno de interesse — como fraudes ou doenças raras.